Práctica 2: guía de arranque por roles

Qué hace cada uno, qué archivos son suyos y cómo trabajamos con GitHub y Jupyter.

REGLA Nº 1: NO SE TOCA EL TRABAJO DE OTROS.
Puedes **abrir y usar** lo que haya hecho otro (sus datos, su preprocesado, sus folds, sus resultados), pero **no cambias nada** de su notebook ni de sus archivos. Si ves un fallo o necesitas un cambio, **se lo dices a esa persona** (por el grupo o con un comentario o issue en GitHub) y lo cambia ella. Si necesitas su código, lo **copias a tu notebook** y lo usas allí, sin editar el suyo.

Reglas para todos

- **Todo lo entregable en inglés**: código, comentarios, gráficas y explicaciones (en celdas Markdown, no en comentarios).
- Primera celda de cada notebook: `PROJECT_RANDOM_STATE = 2627` y un print. La semilla solo se pasa donde hay aleatoriedad.
- **Prohibido entrenar con** `row_id`, `feature_group`, `config_group` o `fold_id`. `iops` ni siquiera viene en los datos.
- Escaladores y encoders **siempre dentro de un** `Pipeline`, para que se ajusten solo con el trozo de entrenamiento.
- En storage, `raid` parece un número pero **es categórica** ("4+2" es un tipo de RAID, no una cantidad): va con one-hot.
- Si usáis IA, apuntadlo en `AI_USAGE.md`: herramienta, para qué, qué partes y cómo lo comprobasteis.

Lo que nos ha dado el profe (y dónde está)

Carpeta	Qué hay
`datos/cyber/`	Datos del reto A: 34.928 filas, 11 columnas para predecir y `attack_code` (10 clases; ransomware solo tiene **208**). También `cyber_groups.csv`, `cyber_class_codes.json` y `PROVENANCE.md`.
`datos/storage/`	Datos del reto B: 12.190 filas (2.438 configuraciones × 5 réplicas), 9 columnas y `latency_ns`. También `storage_groups.csv` y `PROVENANCE.md`.
`plantillas/`	Plantillas que hay que rellenar: `features.md`, `roles.md`, `AI_USAGE.md`, `comparativa.csv`, `informe.md` (el memo) y `model_lock.md` (para la P5).
`herramientas/`	`validate_submission.py`: comprueba el formato de los archivos de entrega.
`esquemas/`, `final_test/`	Formato de entrega explicado; `final_test/` está vacía hasta la P5 (es normal).
`notebooks/`	Vacía: aquí van nuestros notebooks.

Los folds oficiales (`cyber_folds.csv`, `storage_folds.csv`) **todavía no están**: el profe los da a mitad de la P2.2.

Cómo trabajamos: GitHub + Jupyter

Los notebooks de Jupyter se llevan fatal con los merges: si dos personas tocan el mismo `.ipynb`, git no sabe mezclarlo y se rompe. Por eso **cada uno trabaja en su propio notebook** y al final R5 los junta en los 2 notebooks de entrega.

- **Repositorio privado** en GitHub (los datos de cyber son solo para uso académico). R5 lo crea con los archivos del profe y os invita.
- **Una rama por persona**: `r1-datos`, `r2-cyber`, `r3-storage`, `r4-evaluacion`, `r5-informe`. Nadie trabaja directamente en `main`.
- **Notebook de trabajo personal** en `notebooks/borradores/` (por ejemplo `R2_cyber.ipynb`). Es tuyo y solo lo tocas tú.
- Para subir algo: `git pull` de `main` → trabajas en tu rama → commit → **pull request** a `main`. Otro miembro le echa un ojo y se mergea.
- Antes de hacer commit de un borrador: **Kernel** → **Restart & Clear Output**, para que el archivo pese poco y no dé conflictos.
- Si alguien necesita algo tuyo (por ejemplo el preprocesado de R1), lo coge de `main` y lo **copia** a su notebook. No lo edita.

- Al final, R5 crea `notebooks/P2_cyber.ipynb` y `notebooks/P2_storage.ipynb` pegando las secciones de cada uno **sin cambiar su contenido**, y los ejecuta enteros con las salidas guardadas. Si algo falla, avisa al dueño de esa sección.
- Añadir un `.gitignore` con `.ipynb_checkpoints/` y `__pycache__/`.

Orden de trabajo

Fase 1 (a la vez): R5 crea el repo y la estructura, R1 prepara los datos y R4 monta los folds. **Fase 2:** R2 y R3 entrenan modelos usando lo de R1 y R4 (pueden ir explorando antes). **Fase 3:** R5 junta todo, escribe el memo y vuelve a ejecutar los notebooks enteros.

R1 · Datos y preprocesado

Nombre: _____

Te encargas de: P2.1 en los dos retos, el preprocesado común y `features.md`.

Tus archivos (solo los tocas tú): `notebooks/borradores/R1_datos.ipynb` y `features.md`.

Cómo empezar

- **1.** Leer los `PROVENANCE.md` de `datos/cyber/` y `datos/storage/` (en la defensa preguntan de dónde salen los datos y qué se quitó).
- **2.** Cargar `cyber_development.csv` y `storage_development.csv` y comprobar las columnas contra el enunciado: cyber tiene 11 (`proto, service, conn_state` categóricas y 8 numéricas); storage tiene 9 (`load_type, io_type, raid, device_type` categóricas y 5 numéricas).
- **3.** Contar: ejemplos por clase en cyber (con nombres del `.json` y el número de ransomware), número de `feature_group` distintos, número de `config_group` distintos y réplicas por configuración.
- **4.** Crear el preprocesado para los demás: un `ColumnTransformer` con one-hot en las categóricas; en cyber, `log1p` + `StandardScaler` en las numéricas; en storage, `StandardScaler` en las numéricas.
- **5.** Rellenar `features.md` a partir de la plantilla (una fila por columna, con fecha). La plantilla ya trae ejemplos.

Entregas

Sección P2.1 de los dos retos, bloque de preprocesado listo para copiar y `features.md`.

Dependes de / te esperan

No dependes de nadie, así que **empieza el primero**. R2, R3 y R4 necesitan tu preprocesado.

R4 · Evaluación y validación

Nombre: _____

Te encargas de: P2.2 (los folds), las funciones de métricas y la tabla comparativa. Puedes vetar cualquier evaluación que se salte las reglas.

Tus archivos (solo los tocas tú): `notebooks/borradores/R4_evaluacion.ipynb`, `comparativa.csv` y los CSV de folds oficiales cuando lleguen.

Cómo empezar

- **1.** Crear los 5 folds: `StratifiedGroupKFold` con `groups=feature_group` en cyber y `GroupKFold` con `groups=config_group` en storage.
- **2.** Comprobar con código e imprimir que ningún grupo aparece en train y validación a la vez y que todas las clases de cyber salen en cada fold (lo pide el memo).
- **3.** Comparación fija: KNN con `n_neighbors=5` y el preprocesado de R1. Puntuarlo con folds agrupados y con folds normales (`KFold` en storage, `StratifiedKFold` en cyber). Solo puede cambiar el reparto. Explicar por qué en storage la versión normal sale más optimista.
- **4.** Cuando lleguen `cyber_folds.csv` y `storage_folds.csv`, subirlos a `datos/`, compararlos con los vuestros y **a partir de ahí usar solo los oficiales**.
- **5.** Preparar una función que recorra los folds oficiales y calcule las métricas (macro F1; MAE, RMSE y R2 sobre z; MAE en ns) para que la copien R2 y R3.

Entregas

Sección P2.2 de los dos retos, función de evaluación común y filas de `comparativa.csv` (la plantilla trae el formato).

Dependes de / te esperan

Necesitas el preprocesado de R1. R2 y R3 esperan tu función de folds y métricas para dar los resultados finales.

R2 · Clasificación (reto A, cyber)

Nombre: _____

Te encargas de: P2.4 para cyber, P2.5 (ransomware y umbrales) y el CSV de prueba de cyber.

Tus archivos (solo los tocas tú): `notebooks/borradores/R2_cyber.ipynb` y `entregas_prueba/cyber_dry_run.csv`.

Cómo empezar

- **1.** Mientras R4 termina, explorar los datos de cyber y preparar el código. Los números finales se sacan con los folds oficiales.
- **2.** Baseline: `DummyClassifier(strategy="most_frequent")`.
- **3.** KNN con una lista de vecinos que elijáis. Gráfica de macro F1 según el número de vecinos, con media y variación entre folds. Justificar el escalado.
- **4.** Para el KNN elegido: macro F1, weighted F1, accuracy, precisión, recall y F1 por clase junto a su número de ejemplos, y matriz de confusión con los nombres de `cyber_class_codes.json`. Comentar qué clases confunde.
- **5.** P2.5: `predict_proba` en cada fold. Para ransomware frente al resto: ROC, AUC y Brier. Regla con τ = 0.05, 0.10, 0.20, 0.35 y 0.50, más el argmax normal: recall, precisión y falsas alarmas de cada uno (recuerda: ransomware tiene 208 ejemplos).

Entregas

Secciones P2.4 y P2.5 de cyber y un CSV de prueba con `row_id,attack_code` hecho con un trozo de los datos de desarrollo.

Dependes de / te esperan

Necesitas el preprocesado de R1 y los folds y métricas de R4. R5 espera tus gráficas y conclusiones para el memo.

R3 · Regresión (reto B, storage)

Nombre: _____

Te encargas de: P2.3 (justificar el log10), P2.4 para storage y el CSV de prueba de storage.

Tus archivos (solo los tocas tú): `notebooks/borradores/R3_storage.ipynb` y `entregas_prueba/storage_dry_run.csv`.

Cómo empezar

- **1.** Mirar la distribución de `latency_ns` (va de ~$1{,}2 \cdot 10^5$ a ~$2{,}5 \cdot 10^9$ ns): histograma normal y en escala log.
- **2.** P2.3: `DummyRegressor` y un KNN con la latencia tal cual (solo esos dos). Después, lo mismo con `z = log10(latency_ns)`.
- **3.** Escribir el argumento a favor del log: escala de los datos, que el error sea relativo y no absoluto, y la estructura del problema. Nada de “porque sale mejor”. Este texto va tal cual a la defensa.
- **4.** P2.4: KNN con una lista de vecinos que elijáis, sobre z. Gráfica completa con la variación entre folds. Métricas: MAE(z), RMSE(z), R2(z) y MAE en ns (primero se calcula 10^z predicha y luego el error; nunca se eleva a 10 el MAE).
- **5.** Elegir los vecinos que salgan mejor en storage, no copiar los de cyber (eso penaliza).

Entregas

Secciones P2.3 y P2.4 de storage y un CSV de prueba con `row_id,latency_ns` en nanosegundos (no z) y valores positivos.

Dependes de / te esperan

Necesitas el preprocesado de R1 y los folds y métricas de R4. R5 necesita tu justificación del log para el memo.

R5 · Informe y reproducibilidad

Nombre: _____

Te encargas de: El repo, la estructura, los .md del equipo, el memo, la pregunta de OneR y los 2 notebooks finales.

Tus archivos (solo los tocas tú): `notebooks/P2_cyber.ipynb`, `notebooks/P2_storage.ipynb`, `notebooks/borradores/R5_informe.ipynb`, `roles.md`, `AI_USAGE.md`, el memo y `.gitignore`.

Cómo empezar

- **1.** Crear el repo privado con los archivos del profe, el `.gitignore` y las carpetas `notebooks/borradores/` y `entregas_prueba/`. Invitar a todos y crear las 5 ramas.
- **2.** Rellenar `roles.md` (nombres S1 a S5) y `AI_USAGE.md` a partir de las plantillas.
- **3.** Responder la pregunta de OneR de P2.6: por qué no sirve tal cual con columnas mixtas y qué decisiones habría que tomar.
- **4.** Pasar los CSV de prueba de R2 y R3 por `herramientas/validate_submission.py` (en Windows, `python` en vez de `python3`).
- **5.** Escribir el memo con la plantilla `plantillas/informe.md`. Añadir las 8 citas de TON_IoT que pide `datos/cyber/PROVENANCE.md`. Al final, juntar las secciones en los 2 notebooks finales **sin cambiar su contenido**, ejecutar todo de principio a fin y revisar que esté en inglés.

Entregas

Repo y estructura, `roles.md`, `AI_USAGE.md`, la respuesta de OneR, el memo y los 2 notebooks finales ejecutados.

Dependes de / te esperan

Empiezas el primero con el repo. Para el memo y los notebooks finales necesitas a todos, así que márcales fechas.

Ojo: en la defensa el 60% de la nota es individual y os pueden preguntar cualquier parte. Antes de entregar, que cada uno lea y entienda las secciones de los demás (leer sí, tocar no).